

Gramática Livre de Contexto

Trabalho de Compiladores — Parte 2: Análise Sintática

Método de análise: **bottom-up LR (SLR(1))**

1. Definição formal

A gramática é a quádrupla $G = (N, T, P, S)$, com símbolo inicial $S = \langle \text{programa} \rangle$.

Após o aumento da gramática com a produção $\langle S' \rangle \rightarrow \langle \text{programa} \rangle$, a construção da coleção canônica LR(0) produz **116 estados** e a tabela SLR(1) resultante tem **761 entradas em ACTION**, **sem nenhum conflito** shift/reduce ou reduce/reduce. A gramática é, portanto, SLR(1) — e consequentemente também LALR(1) e LR(1).

> **Nota sobre a produção aumentada.** Ela é escrita aqui como $\langle S' \rangle \rightarrow \langle \text{programa} \rangle$, e **não** como $\langle S' \rangle \rightarrow \langle \text{programa} \rangle \$$. O marcador $\$$ não > é um símbolo da parte direita: ele é o *lookahead* que dispara a ação > **aceitar**. Ou seja, o analisador aceita quando o item $\langle S' \rangle \rightarrow \langle \text{programa} \rangle \bullet$ > está no estado do topo e o token corrente é $\$$. É a convenção usada na > implementação — **AUMENTADA** em `tabela_slr.py` tem parte direita > $(\langle \text{programa} \rangle,)$, e o $\$$ aparece apenas em `ACTION[estado, \$] = acc`. As duas > convenções (com e sem o $\$$ na produção) são equivalentes e ambas aparecem na > literatura; adotamos uma só para que documento e código não divirjam.

2. Conjunto de terminais (T)

Os 34 terminais correspondem exatamente ao campo `terminal` produzido pelo analisador léxico (`analisador_lexico.py`). O marcador de fim de entrada $\$$ é listado junto por conveniência, mas **não faz parte de T**: ele não aparece no lado direito de nenhuma produção, existindo apenas como coluna da tabela ACTION, onde dispara a ação de aceitação (ver a nota da seção 1).

Palavras reservadas

Terminal	Lexema
<code>int</code>	<code>int</code>
<code>float</code>	<code>float</code>
<code>string</code>	<code>string</code>
<code>bool</code>	<code>bool</code>
<code>maybe</code>	<code>maybe</code>
<code>default</code>	<code>default</code>
<code>cycle</code>	<code>cycle</code>
<code>walk</code>	<code>walk</code>
<code>rd</code>	<code>rd</code>
<code>wt</code>	<code>wt</code>

Operadores

Terminal	Lexema(s)	Categoria
atrib	=	atribuição
igual	==	relacional
diferente	!=	relacional
menor	<	relacional
menor_igual	<=	relacional
maior	>	relacional
maior_igual	>=	relacional
mais	+	aritmético
menos	-	aritmético (binário e unário)
vezes	*	aritmético
dividido	/	aritmético
e_logico	&& ou AND	lógico
ou_logico	// ou OR	lógico
nao_logico	! ou NOT	lógico

As duas grafias de cada operador lógico produzem o **mesmo terminal**, para não duplicar produções na gramática.

Delimitadores

Terminal	Lexema
abre_par	(
fecha_par	)
abre_chave	{
fecha_chave	}
ponto_virgula	;
virgula	,

Símbolos com atributo e fim de entrada

Terminal	Descrição
id	identificador: <code>[a-zA-Z_][a-zA-Z0-9_]*</code>
num_int	literal inteiro: <code>[0-9]+</code>
num_float	literal real: <code>[0-9]+.[0-9]+</code>
lit_string	literal de cadeia entre aspas duplas, sem quebra de linha
\$	marcador de fim de entrada

Comentários de linha (`# ...`) e de bloco (`/* ... */`) são descartados pelo analisador léxico e **não geram terminais**.

3. Produções (P)

As produções estão numeradas: o número é o argumento da ação **reduzir** na tabela SLR(1). ϵ denota a produção vazia.

3.1 Estrutura do programa

```
(0)  <S'>           → <programa>
(1)  <programa>      → <lista_comandos>
(2)  <lista_comandos> → <lista_comandos> <comando>
(3)  <lista_comandos> →  $\epsilon$ 
(4)  <comando>       → <declaracao>
(5)  <comando>       → <atribuicao>
(6)  <comando>       → <leitura>
(7)  <comando>       → <escrita>
(8)  <comando>       → <condicional>
(9)  <comando>       → <repeticao_cond>
(10) <comando>       → <repeticao_cont>
(11) <comando>       → <bloco>
(12) <bloco>         → abre_chave <lista_comandos> fecha_chave
```

A recursão à esquerda em `<lista_comandos>` é intencional: em um analisador ascendente ela mantém a pilha em profundidade constante, ao contrário da recursão à direita.

3.2 Declarações

```
(13) <declaracao>    → <tipo> <lista_decl> ponto_virgula
(14) <tipo>          → int
(15) <tipo>          → float
(16) <tipo>          → string
(17) <tipo>          → bool
(18) <lista_decl>     → <lista_decl> virgula <item_decl>
(19) <lista_decl>     → <item_decl>
(20) <item_decl>      → id
(21) <item_decl>      → id atrib <expressao>
```

Permite `int a;`, `int a = 0;`, `int a, b, c;` e `int a = 1, b;`.

3.3 Atribuição

```
(22) <atribuicao>     → id atrib <expressao> ponto_virgula
```

3.4 Comandos de entrada e saída

```

(23) <leitura>      → rd abre_par <lista_id> fecha_par ponto_virgula
(24) <lista_id>     → <lista_id> virgula id
(25) <lista_id>     → id

(26) <escrita>      → wt abre_par <lista_expr> fecha_par ponto_virgula
(27) <lista_expr>   → <lista_expr> virgula <expressao>
(28) <lista_expr>   → <expressao>

```

`rd` recebe apenas identificadores (destinos de escrita); `wt` aceita qualquer expressão.

3.5 Comando condicional

```

(29) <condicional>  → maybe abre_par <expressao> fecha_par <bloco>
(30) <condicional>  → maybe abre_par <expressao> fecha_par <bloco>
                        default <bloco>

```

As duas alternativas são escritas **explicitamente**, em vez de um não-terminal opcional `<senao> → default <bloco> | ε`. A forma com ϵ introduziria um conflito shift/reduce no estado que contém o item `<condicional> → maybe abre_par <expressao> fecha_par <bloco> •`.

Além disso, como o corpo do `maybe` é obrigatoriamente um `<bloco>` entre chaves, **o problema do dangling else não existe** nesta linguagem: um `default` nunca pode se ligar ambigualmente a dois `maybe` diferentes.

3.6 Comandos de repetição

```

(31) <repeticao_cond> → cycle abre_par <expressao> fecha_par <bloco>

(32) <repeticao_cont> → walk abre_par <inicializacao> ponto_virgula
                        <expressao> ponto_virgula
                        <passo> fecha_par <bloco>

(33) <inicializacao> → <tipo> id atrib <expressao>
(34) <inicializacao> → id atrib <expressao>

(35) <passo>         → id atrib <expressao>

```

`cycle` é a repetição condicional (while); `walk` é a repetição contada (for), que aceita tanto `walk (int i = 0; ...)` quanto `walk (i = 0; ...)`.

3.7 Expressões

A precedência e a associatividade estão embutidas na **estratificação dos não-terminais**, e não em declarações externas de precedência. Isso torna a gramática não ambígua por construção, sem depender de declarações de precedência resolvidas por uma ferramenta externa.

Da menor para a maior precedência:

```

(36) <expressao>      → <expressao> ou_logico <exp_e>
(37) <expressao>      → <exp_e>

(38) <exp_e>          → <exp_e> e_logico <exp_nao>
(39) <exp_e>          → <exp_nao>

(40) <exp_nao>        → nao_logico <exp_nao>
(41) <exp_nao>        → <exp_rel>

(42) <exp_rel>        → <exp_arit> <op_rel> <exp_arit>
(43) <exp_rel>        → <exp_arit>

(44) <op_rel>         → igual
(45) <op_rel>         → diferente
(46) <op_rel>         → menor
(47) <op_rel>         → menor_igual
(48) <op_rel>         → maior
(49) <op_rel>         → maior_igual

```

```

(50) <exp_arit>       → <exp_arit> mais <termo>
(51) <exp_arit>       → <exp_arit> menos <termo>
(52) <exp_arit>       → <termo>

(53) <termo>          → <termo> vezes <fator>
(54) <termo>          → <termo> dividido <fator>
(55) <termo>          → <fator>

(56) <fator>          → abre_par <expressao> fecha_par
(57) <fator>          → menos <fator>
(58) <fator>          → id
(59) <fator>          → num_int
(60) <fator>          → num_float
(61) <fator>          → lit_string

```

Tabela de precedência resultante:

Nível	Operadores	Associatividade
1 (menor)	// OR	esquerda
2	&& AND	esquerda
3	! NOT	direita (prefixo)
4	== != < <= > >=	não associativo
5	+ - (binário)	esquerda
6	* /	esquerda
7 (maior)	- (unário), ()	direita (prefixo)

O nível 4 é **não associativo** por construção: a produção (42) usa <exp_arit> dos dois lados, e não <exp_rel>. Isso rejeita $a < b < c$, que não teria significado útil na linguagem, e elimina uma fonte de ambiguidade.

O menos unário (57) está no nível de <fator>, o mais alto, de modo que $-a * b$ é lido como $(-a) * b$.

4. Conjunto de não-terminais (N)

<programa>	<lista_comandos>	<comando>	<bloco>
<declaracao>	<tipo>	<lista_decl>	<item_decl>
<atribuicao>	<leitura>	<lista_id>	<escrita>
<lista_expr>	<condicional>	<repeticao_cond>	<repeticao_cont>
<inicializacao>	<passo>	<expressao>	<exp_e>
<exp_nao>	<exp_rel>	<op_rel>	<exp_arit>
<termo>	<fator>		

Total: 26 não-terminais.

5. Tratamento de erros sintáticos (modo pânico)

O analisador não para no primeiro erro: registra o defeito, sincroniza e segue, de modo que uma única compilação reporte tantos erros quantos for possível.

5.1 A sincronização é feita por não-terminais

A recuperação **não** parte de uma lista de terminais escolhidos à mão: ela parte de **não-terminais de sincronização**, e os terminais em que a análise recomeça são consequência — são os conjuntos FOLLOW desses não-terminais, calculados pelo mesmo código que monta a tabela ([tabela_slr.py](#)).

São três, tentados nesta ordem (constante `SINCRONIZADORES`, em [analisador_sintatico.py](#)):

Ordem	Não-terminal	Efeito da escolha
1	<comando>	descarta o comando defeituoso inteiro e retoma no próximo — é o reparo mais robusto
2	<expressao>	reparo mais local, que preserva mais da árvore; só funciona quando o erro está de fato dentro de uma expressão
3	<lista_comandos>	último recurso, no nível mais externo

Os FOLLOW correspondentes, que são os pontos reais de retomada:

```

FOLLOW(<comando>)      = { $   {   }   int   float   string   bool
                        id   maybe   cycle   walk   rd   wt }

FOLLOW(<lista_comandos>) = { $   {   }   int   float   string   bool
                        id   maybe   cycle   walk   rd   wt }

FOLLOW(<expressao>)     = { )   ;   ,   // }
```

Repare que **;** **não** está em `FOLLOW(<comando>)`: um `<comando>` já termina com **;** (produções 13, 22, 23, 26), então nada que o siga pode ser **;**. O ponto e vírgula sincroniza pela via de `<expressao>` — é o caso de `contador = contador + ;`, em que o erro está dentro da expressão e a retomada acontece no próprio **;**.

5.2 O algoritmo

1. Ao encontrar uma célula vazia em `ACTION[estado, terminal]`, registra-se o erro com **linha e coluna** do token e a lista de terminais que teriam ação definida naquele estado.
2. Para cada não-terminal de sincronização **A**, na ordem da tabela acima, percorre-se a pilha **do topo para a base** procurando um estado **s** com `GOTO[s, A]` definido.

3. Encontrado **s**, descartam-se tokens da entrada até um que esteja em **FOLLOW(A)**. Se a entrada acabar antes disso, tenta-se o **A** seguinte.
4. Desempilha-se até **s**, empilha-se **GOTO[s, A]** e a análise prossegue dali. Os símbolos descartados da pilha viram um nó de **A** marcado como **<trecho com erro>** na árvore sintática, de modo que a árvore continua completa mesmo com erros.
5. Se nenhum **A** servir, descarta-se um token e tenta-se de novo.

5.3 Controle de cascata

Um único defeito pode gerar várias mensagens. Duas regras evitam isso:

- **Uma mensagem por posição da entrada.** A recuperação pode falhar e reincidir sobre o mesmo token; como a posição na entrada nunca retrocede, basta comparar com a última posição reportada.
- **Mínimo de deslocamentos entre mensagens.** Depois de um erro reportado, é preciso deslocar **MIN_DESLOCAMENTOS = 2** tokens com sucesso antes de reportar o próximo. Enquanto o analisador não voltou a andar sobre entrada válida, um novo erro é mais provavelmente eco da recuperação do que defeito novo. Em **a + b;**, a sincronização retoma em **b**, que abre um comando que também não se completa — o limiar mata essa segunda mensagem.

Essa é uma heurística clássica de recuperação de erros em análise ascendente, descrita na literatura (Aho, Lam, Sethi & Ullman) e **reimplementada aqui em quatro linhas** — um contador de deslocamentos desde o último erro. O limiar mais citado é 3; aqui ele foi calibrado para **2** sobre os casos de teste deste trabalho, porque com 3 defeitos distintos e próximos (duas atribuições vazias seguidas) deixavam de ser reportados.

As mensagens omitidas por essas regras são **contadas e informadas ao final**, para que nada desapareça em silêncio.

5.4 Terminação

Se a recuperação não progredir após duas tentativas no mesmo ponto, um token é descartado à força; e acima de 50 erros a análise é interrompida com aviso. Juntos, os dois limites garantem que o analisador sempre termina, qualquer que seja a entrada.

5.5 Alternativas consideradas

Para analisadores LR existe uma segunda família de tratamento de erros, a do **reparo local**, que consiste em expandir a tabela em duas frentes: nas linhas que contêm reduções, as células em branco são preenchidas com essas reduções (reduções por omissão); nas demais células entram rotinas específicas, que inserem o símbolo que falta ou descartam o símbolo inesperado e deixam a análise seguir.

Ela foi avaliada e descartada em favor da sincronização por não-terminais descrita em 5.1 e 5.2, por três razões:

- **Cobertura.** Rotinas de reparo precisam ser escritas ***célula a célula***, e cada posição da matriz pede um tratamento diferente. Com 116 estados e 34 terminais, um conjunto realista de rotinas cobriria apenas uma fração das células vazias, e o resto ficaria sem recuperação alguma. A sincronização por FOLLOW é uniforme: vale para qualquer célula vazia, sem enumeração manual.
- **Diagnóstico.** O reparo local pode mascarar o defeito: ao inserir o símbolo que falta, o analisador termina ***aceitando*** uma sentença que estava errada. Aqui um programa com erro é sempre rejeitado, e cada defeito é reportado com linha e coluna.
- **Reduções por omissão não antecipam a detecção.** Preencher as células em branco das linhas de redução não faz o analisador aceitar nada a mais: ele apenas executa reduções antes de perceber o erro, que continua sendo detectado na mesma posição da entrada, porque nenhum token inválido chega a ser empilhado. O ganho é de tamanho de tabela, não de qualidade do diagnóstico.

Da abordagem de reparo local foi aproveitado o **formato da mensagem**: nomear o token inesperado e listar os terminais que seriam aceitos naquele estado.

5.6 Verificação passo a passo

A construção pode ser inspecionada em cada etapa, o que permite conferir a tabela e o reconhecimento à mão:

Etapa	Onde está
Gramática aumentada	seção 3.1 deste documento
Conjunto de itens (autômato LR(0)), <code>I0..I115</code>	<code>--itens</code>
Tabela ACTION/GOTO (<code>E<n></code> , <code>R<n></code> , <code>AC</code>)	<code>--tabela</code>
Tratamento de erros	seções 5.1 a 5.5
Reconhecimento da sentença (Pilha \	Entrada \ Ação) <code>--passos</code>

Uma observação sobre a última: a apresentação usual do algoritmo LR usa **uma** pilha, com estados e símbolos intercalados, desempilhando `2·|β|` a cada redução. O código usa **duas pilhas paralelas** (`pilha_estados` e `pilha_nos`), desempilhando `|β|` de cada uma. A informação é a mesma, e manter os nós em pilha separada é o que permite montar a árvore sintática; a saída de `--passos` entrelaça as duas para exibí-las na forma intercalada.

5.7 Consequência sobre o analisador léxico

É a sincronização acima que exigiu corrigir o léxico para emitir um **terminal distinto por símbolo** (`ponto_virgula`, `fecha_chave`, `abre_chave`, ...). Na versão da Parte 1 todos compartilhavam a classe `simbolo_simples`, e sem terminais distintos não haveria como calcular FOLLOW útil nem distinguir os pontos de retomada na tabela.

6. Verificação

A gramática foi verificada por um construtor de tabela SLR(1) escrito para este trabalho:

- **116** estados na coleção canônica LR(0);
- **0** conflitos shift/reduce; **0** conflitos reduce/reduce;
- a tabela obedece à definição de SLR(1): toda ação de empilhamento corresponde a uma transição do autômato, e cada redução por $A \rightarrow \alpha$ existe exatamente nos estados que contêm o item $A \rightarrow \alpha \cdot$ e apenas para os lookaheads de `FOLLOW(A)` — verificado célula a célula sobre as 761 entradas;
- a aceitação ocorre em um único estado, o que contém $\langle S' \rangle \rightarrow \langle \text{programa} \rangle \cdot$.

Cobertura do programa de teste. `programa_teste.txt` exercita:

- as **61** produções da gramática (todas, exceto a aumentada, que não é reduzida);
- os **34** terminais;
- as duas grafias de cada operador lógico (`&&/AND`, `///OR`, `!/NOT`), os seis relacionais, os quatro aritméticos e o menos unário;

- os quatro tipos, declaração com e sem inicialização e em lista, `rd` e `wt` com um e com vários argumentos, `maybe` com e sem `default`, `cycle`, `walk` nas duas formas de inicialização, bloco solto e comentários de linha e de bloco.

Verificação da derivação. As reduções produzidas pelo analisador, lidas de trás para frente, foram aplicadas ao símbolo inicial substituindo sempre o não-terminal mais à direita: as 420 reduções regeneram exatamente os 202 tokens da entrada. Isso confirma na prática que a sequência de reduções é uma derivação mais à direita percorrida na ordem inversa.

Rejeição. Entradas inválidas são corretamente rejeitadas, entre elas: ponto e vírgula ausente, chave não fechada, `maybe` sem bloco, `default` sem `maybe` correspondente, relacional encadeado (`a < b < c`) e expressão usada como comando (`a + b;`).

7. Observações sobre a linguagem

Pontos deliberados desta definição, que podem ser revistos:

- A vírgula foi incorporada ao alfabeto na Parte 2. Sem ela não haveria como escrever listas de declaração nem `wt` com mais de um argumento.
- O tipo `bool` existe, mas a linguagem não possui literais `true/false`: um valor booleano só é obtido a partir de uma expressão relacional ou lógica.
- Não há vetores (`[ ]`), operador de módulo (`%`), nem declaração de funções ou procedimentos.
- A verificação de tipos (por exemplo, atribuir `lit_string` a uma variável `int`) não é tratada aqui: é responsabilidade da análise semântica, e a gramática livre de contexto, por definição, não pode expressá-la.